Project - High Level Design

On

Legal Document QA System

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Jogendar Das Bairagi	EN22ME304045
2.	Roshan Bankar	EN22EL301072
3.	Vivek Gangrade	EN22CS3011098
4.	Maahi Maheshwari	EN22ME304057
5.	Shubhangi Prajapat	EN22ME304094

Group Name : 04D10

Project Number : DO-17

Industry Mentor Name : Mr. Yash Mittal

University Mentor Name : Prof. Divya Kumawat

Academic Year : 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. **References**

4. **Interfaces**

5. **State and Session Management**

6. **Caching**

7. **Non-Functional Requirement**
   Security Aspects

8. **References**

1. Introduction

1.1. Scope of the document

This document defines the high-level architecture of the Legal-AI Assistant. It covers the structural design, data management, and the integration of Large Language Models (LLMs) with local vector storage.

1.2. Intended Audience

This document is intended for individuals involved in the design, development, deployment, and evaluation of the Legal Document QA System.The primary audience includes:

- Software Developers and Engineers who will implement and maintain the system architecture.
- Engineers responsible for integrating Large Language Models and vector databases for intelligent responses.
- Researchers and Students studying Generative AI applications in education and non-profit domains.

The document provides both technical and conceptual information to help these audiences understand how the system operates and how its components interact.

1.3. System Overview

The Legal-AI Assistant is an intelligent document analysis platform designed to help users extract precise information from complex legal contracts through a natural language interface.

Instead of manually searching through dense PDFs, users interact with a chatbot that processes document text using a Large Language Model (LLM). When a user asks a question, the system doesn't just "guess"; it retrieves the exact relevant clauses from the uploaded contract and provides a grounded, factual explanation. This ensures that the AI's feedback is always tied to the specific document provided, reducing the risk of legal misinformation.

Key technologies used in the system include:

- Python for backend development
- Large Language Models (LLMs) for intelligent evaluation and explanation

- Vector Databases (FAISS/ChromaDB) for semantic search
- Gradio for building the chatbot interface
- API integrations for connecting the application with AI models

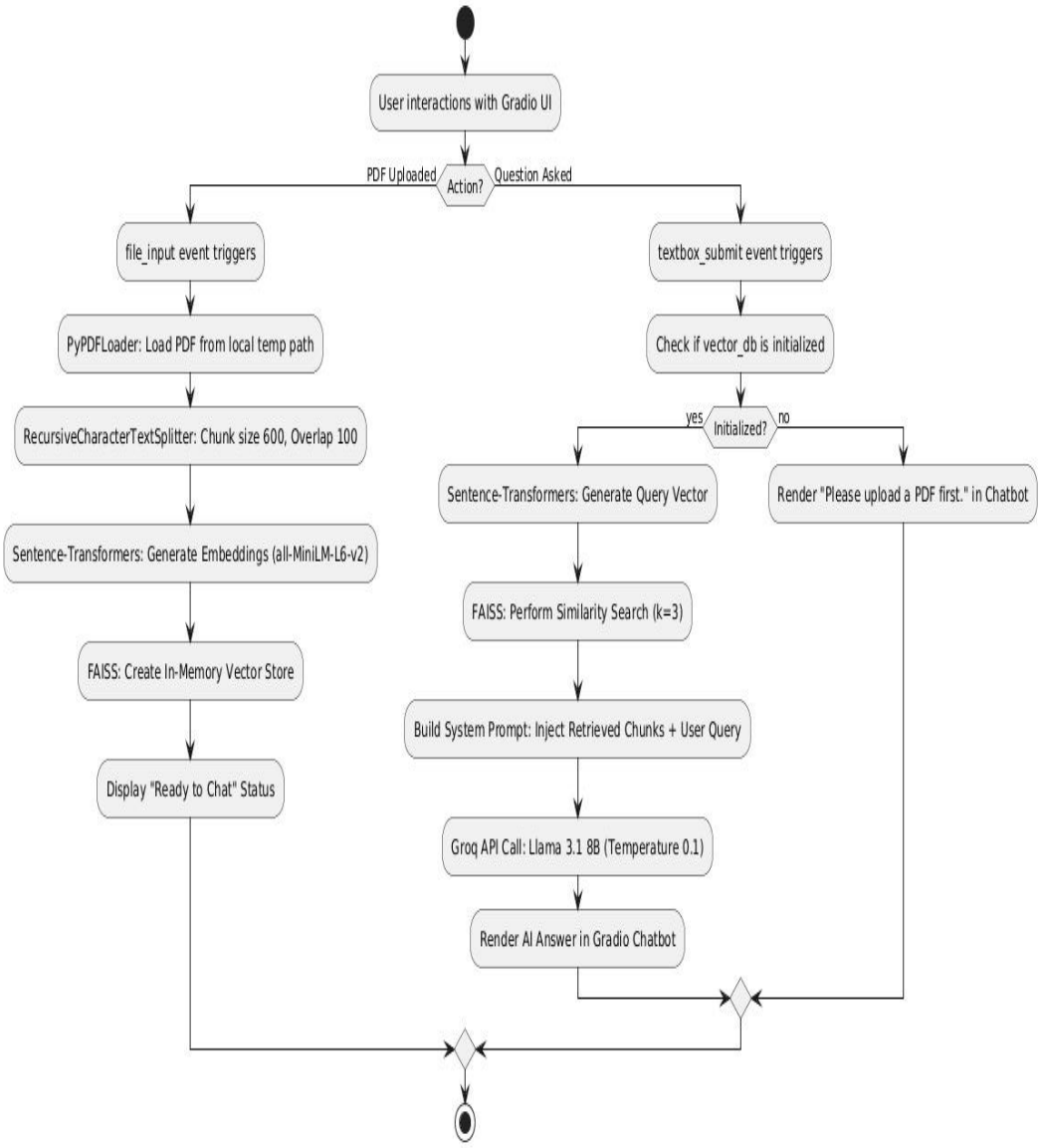
2. System Design

2.1. Application Design

The application follows a **Three-Tier Architecture** (Logic-based):

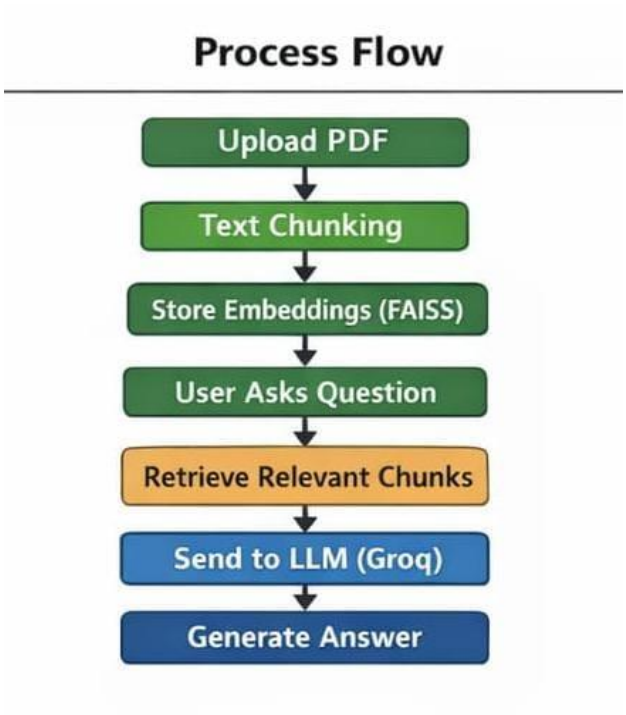
1. **The Interface (Gradio):** A web-based frontend that allows users to upload PDF files and engage in a conversational chat. It handles the "Session State", ensuring the user sees a continuous stream of dialogue.
2. **Intelligence Tier:** An external inference engine that runs the **Llama 3.1** model. It acts as the "Legal Expert," taking the raw data retrieved from the document and turning it into a human-readable answer.
3. **The Knowledge Base (FAISS):** A local vector database that acts as the system's "Instant Memory." It stores the document as mathematical vectors, allowing the system to perform a Semantic Search to find the correct paragraph in milliseconds.

SYSTEM DESIGN



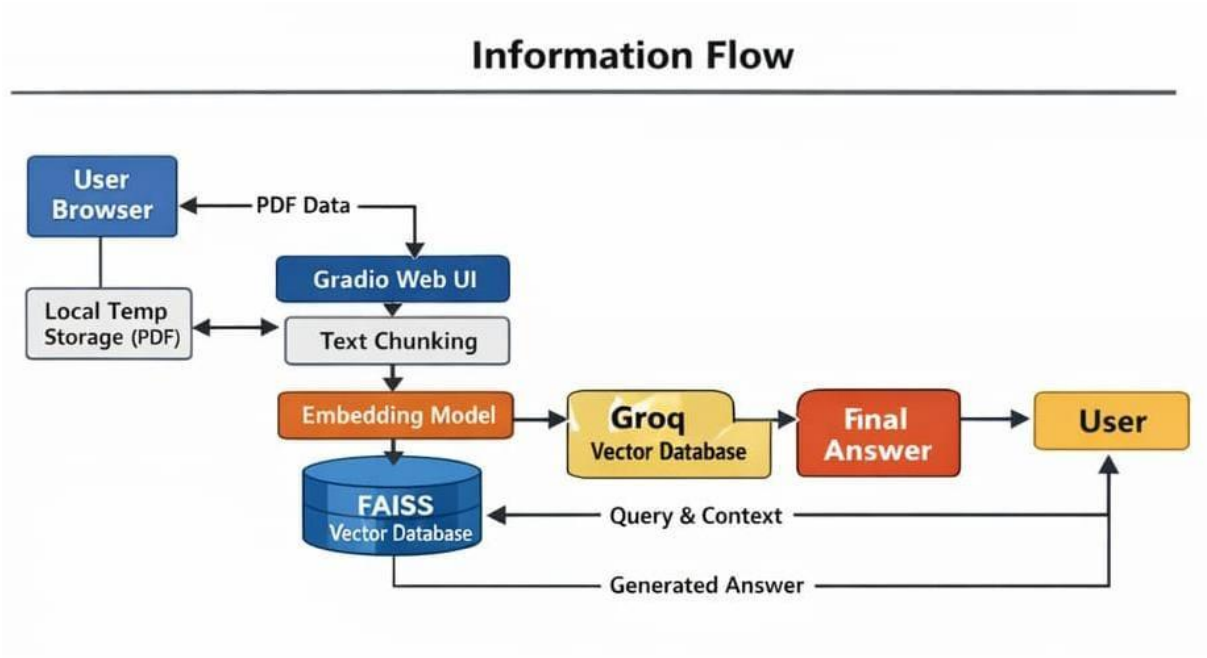
2.2. Process Flow

- **Upload:** User uploads a PDF.
- **Index:** System reads, chunks, and stores text in a vector database.
- **Chat:** User asks a question; system finds relevant text and sends it to the AI.



2.3. Information Flow

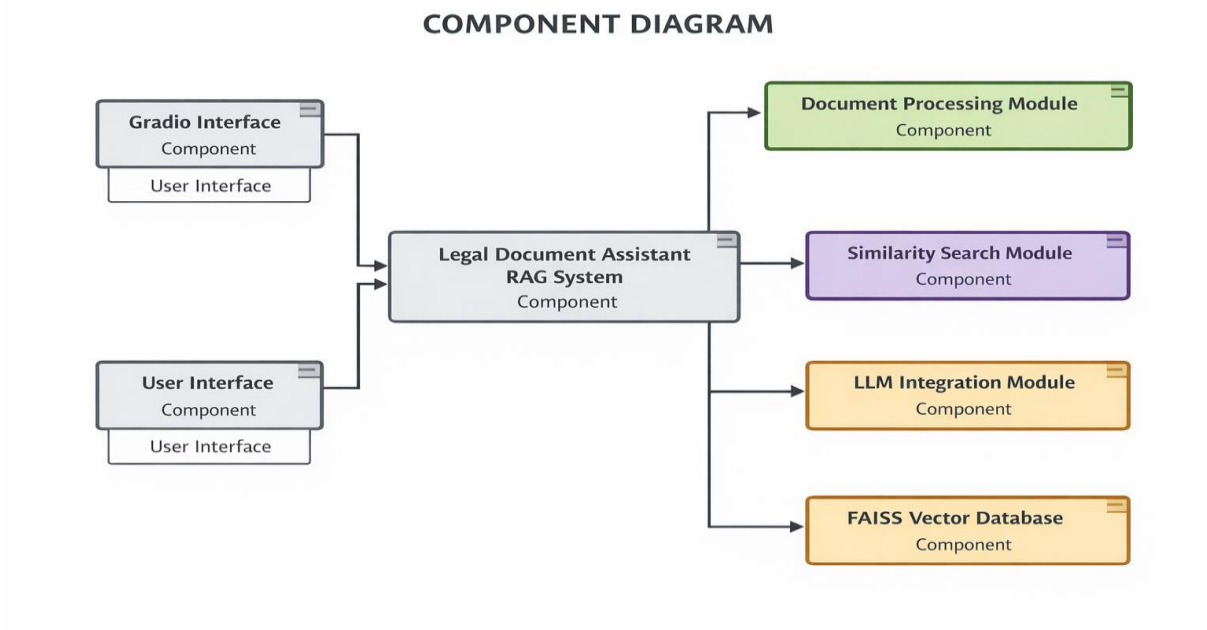
1. **PDF Data:** Moves from the User Browser → Gradio Local Temp Folder.
2. **Vector Data:** Moves from the Embedding Model → In-Memory FAISS Index.
3. **Contextual Prompt:** Moves from the Local Python Environment → External Groq API.



2.4. Components Design

- **Frontend (Gradio):** Handles file buffers and chat state.
- **Orchestrator (LangChain):** Manages the "Chains" (connections) between data and the LLM.
- **Vector Engine (FAISS):** Provides high-speed semantic search capabilities.

Component	Technology	Responsibility
Document Loader	PyPDFLoader	Parsing PDF structure and extracting raw text.
Text Splitter	RecursiveCharacterTextSplitter	Breaking text into 600-character chunks with 100-character overlap.
Embeddings	HuggingFaceEmbeddings	Generating semantic signatures for text chunks.
Vector Store	FAISS	Performing K-Nearest Neighbor (KNN) search for relevant context.
LLM Inference	Groq	Generating natural language answers based on retrieved context.



2.5. Key Design Considerations

- **Accuracy:** Using document retrieval to prevent AI hallucinations.
- **Speed:** Utilizing Groq's high-speed inference.
- **Chunk Overlap (100 chars):** Critical for legal documents to ensure that definitions or clauses split at boundaries remain understandable.
- **Grounding:** The system prompt explicitly restricts the LLM to answer only from the provided context.

2.6. API Catalogue

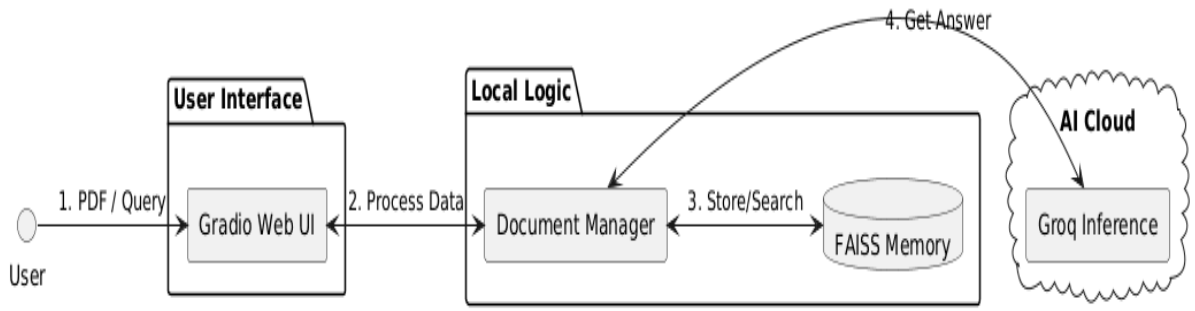
- **FAISS.from_documents()**: Internal API to build the vector index.
- **vector_db.similarity_search()**: Internal API to retrieve relevant document chunks.
- **client.chat.completions.create()**: External API call to the Groq cloud for LLM inference.

3. Data Design

3.1. Data Model

The system processes **unstructured data** from legal PDF documents. The data model follows a transformation pipeline:

- **Raw Input:** PDF blocks extracted using PyPDFLoader.
- **Document Objects:** Text is wrapped into LangChain Document objects containing the text and metadata (like page numbers).
- **Chunks:** Documents are broken into 600-character segments with a 100-character overlap to maintain legal context



3.2. Data Access Mechanism

The system uses Semantic Search via a Retrieval-Augmented Generation (RAG) approach:

- **Vector Search:** When a user asks a question, the question is converted into a vector.
- **Similarity Lookup:** The system uses L2 (Euclidean Distance) to find the top 3 most similar text chunks in the FAISS database.
- **Context Injection:** These 3 chunks are "fed" to the Groq API as the primary source of truth for the answer.

3.3. Data Retention Policies

To ensure privacy for sensitive legal documents, the system follows a Volatile Storage Policy:

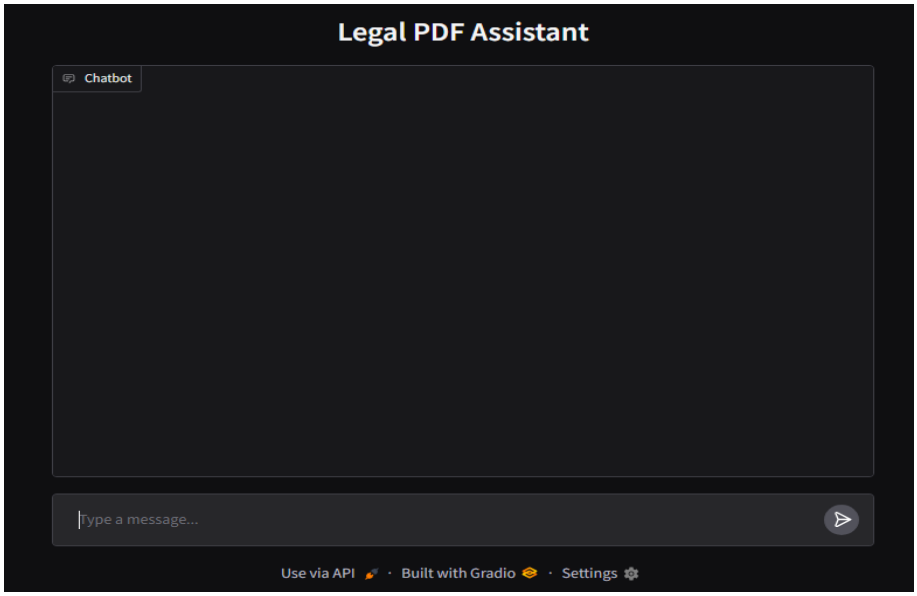
- **In-Memory Storage:** The FAISS index and text chunks are stored in the system's RAM, not on a hard drive.
- **Auto-Purge:** All data is automatically deleted when the Gradio application is closed or the Python script stops running.
- **No Persistence:** No user queries or document contents are permanently stored, ensuring zero data footprint after the session.

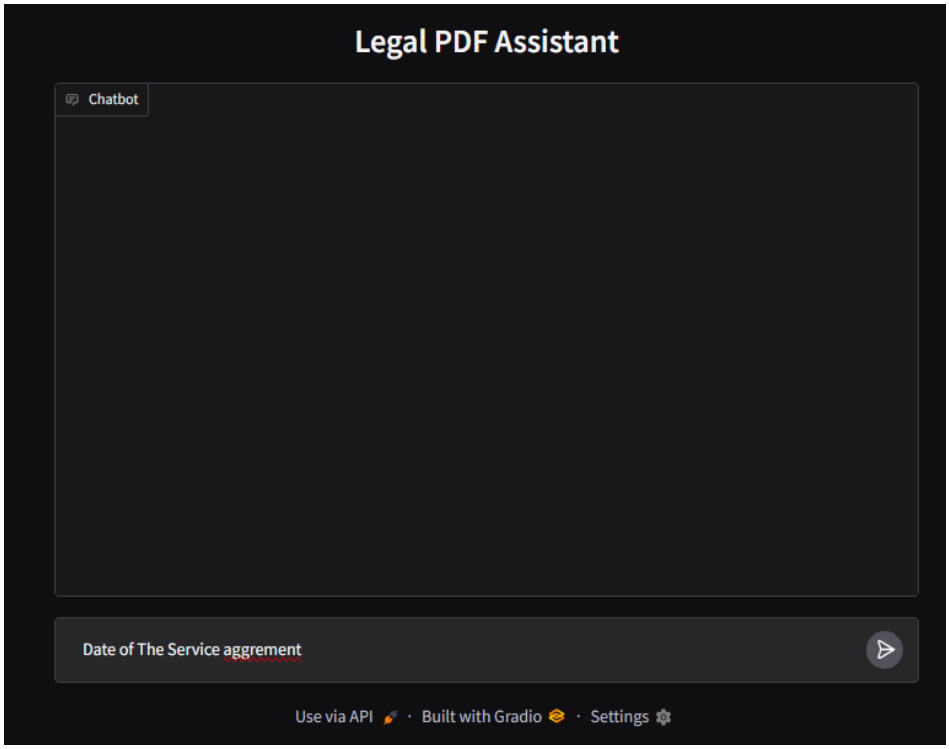
3.4. Data Migration

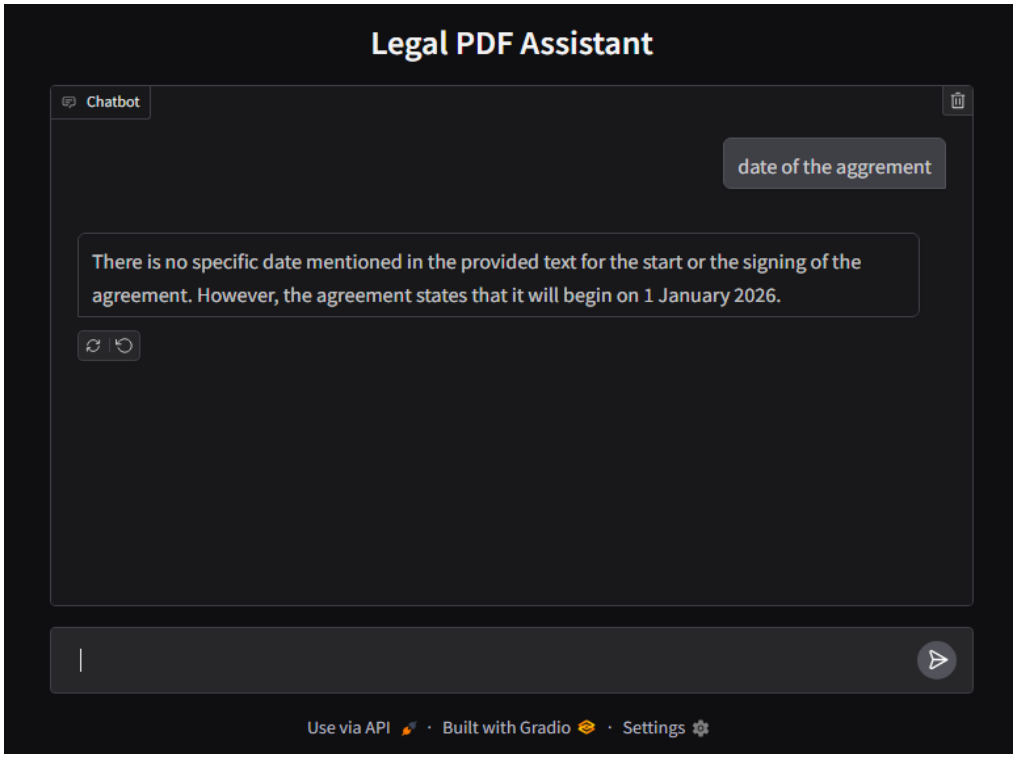
- **Extraction:** Read text from the uploaded PDF.
- **Segmentation:** Split the text into manageable 600-character chunks.
- **Embedding:** Convert each chunk into a mathematical vector.
- **Indexing:** Load these vectors into the **FAISS** local database. This pipeline is triggered instantly upon file upload in the Gradio interface.

4.Interfaces

A web-based Gradio dashboard accessible via localhost:7860.







5. State and Session Management

The application utilizes a **Volatile Session Model** designed for privacy and speed. A user session is initiated the moment a legal PDF is uploaded through the Gradio interface.

- **Client-Side Continuity:** The conversation history and UI state are maintained within the browser's memory. Gradio uses an internal state variable to track the sequence of questions and answers during the active session.
- **Backend Independence:** The Python backend processes each query independently. However, the FAISS Vector Index remains active in the server's RAM as long as the application is running, allowing for multiple follow-up questions on the same document

- **Stateless Architecture:** No persistent databases (like SQL) or authentication systems are used. Once the user refreshes the page or the local server is restarted, all uploaded data and chat logs are automatically purged from the system's memory.

6. Caching

The system implements in-memory caching for the vector database (ChromaDB) to improve performance. The vector store is loaded once during application startup and stored in a global variable, allowing it to be reused across multiple requests without repeated disk access. This reduces latency during semantic search operations and enhances overall system efficiency. However, caching is limited to the vector store, and LLM responses are generated dynamically for each request.

7. Non-Functional Requirements

7.1. Security Aspects

- **API Key Protection:** Secrets are loaded via environment variables.
- **Data Privacy:** Since the Vector DB is in-memory and local, the document text never leaves the local machine except for the specific chunks sent to Groq.
- **Security Scope and Future Enhancements:** Advanced security features like authentication, authorization, and encryption are not implemented and can be considered as future enhancements.

7.2. Performance Aspects

- **Inference Speed:** Groq LPU ensures response times under 2 seconds.
- **Search Speed:** FAISS provides sub-millisecond retrieval for documents under 1,000 pages.
- **Optimized Data Retrieval :** The system shall use Retrieval-Augmented Generation (RAG) to fetch only relevant data, improving efficiency.

8. References

- *Groq LPU Architecture Whitepaper.*
- *LangChain RAG Documentation.*
- *FAISS: A Library for Efficient Similarity Search.*